Experiment # 02

Implementation of Profiling in C using GPROF

Objective:

The objective of this lab is to introduce the profiling and profile the simple code using gprof

to check and observe the execution time taken by each function in an overall code.

Software Tools:

- Ubuntu
- MobaXterm

Theory:

MobaXterm:

MobaXterm is an enhanced terminal for Windows with an X11 server, a tabbed SSH client and several other network tools for remote computing (VNC, RDP, telnet, login). MobaXterm brings all the essential Unix commands to Windows desktop, in a single portable exe file which works out of the box.

Gprof:

Gprof is a performance analysis tool used to profile applications to determine where time is spent during program execution. Gprof is included with most Unix/Linux implementations, is simple to use, and can quickly show which parts of an application take the most time. It can also tell you which functions are being called more or less often than you expected.

How to use gprof:

Using the gprof tool is not at all complex. You just need to do the following on a high-level:

- Have profiling enabled while compiling the code
- Execute the program code to produce the profiling data
- Run the gprof tool on the profiling data file (generated in the step above).

The last step above produces an analysis file which is in human readable form. This file contains a couple of tables (flat profile and call graph) in addition to some other information. While flat profile gives an overview of the timing information of the functions like time consumption for

the execution of a particular function, how many times it was called etc. On the other hand, call graph focuses on each function like the functions through which a particular function was called, what all functions were called from within this particular function etc So this way one can get idea of the execution time spent in the sub-routines too.

Procedure:

First open MobaXterm and login with your given credentials in SSH shell.
Enter the Remote host IP address and your Host username. Click OK
Insert your Password and you are logged in.
After successful login, clear the screen by typing “clear” in the command window.
Right now, you are at your home directory. Which is “home/username”?
Now create a new directory with the “mkdir” for this lab.
It will create the new directory named “lab02”
Now change your directory to lab02 by using “cd” command
Then you have to Paste the example code. Simply drag from your windows directory to the MobaXterm directory Panel.

Example Code:

```
#include <stdlib.h>
#include <stdio.h>
#include <math.h>

void sub(float *  diff, float *  in1, float *  in2, int size)
{
    int i;
    //for(;i<0xffffffff;i++); //some dummy delay

    for (i = 0; i < size; i++)
    {
        diff[i] = in1[i] - in2[i];
    }

}

void add(float *  sum, float *  in1, float *  in2, int size)
{
    int i;
    for (i = 0; i < size; i++)
    {
        sum[i] = in1[i] + in2[i];
    }

    //for(;i<0xffffffff;i++); //some dummy delay

}

void muldiv(float *  prod, float *  qout, float *  in1, float *  in2, int size)
{
    int i;

    for (i = 0; i < size; i++)
    {
        prod[i] = in1[i] * in2[i];
    }
```

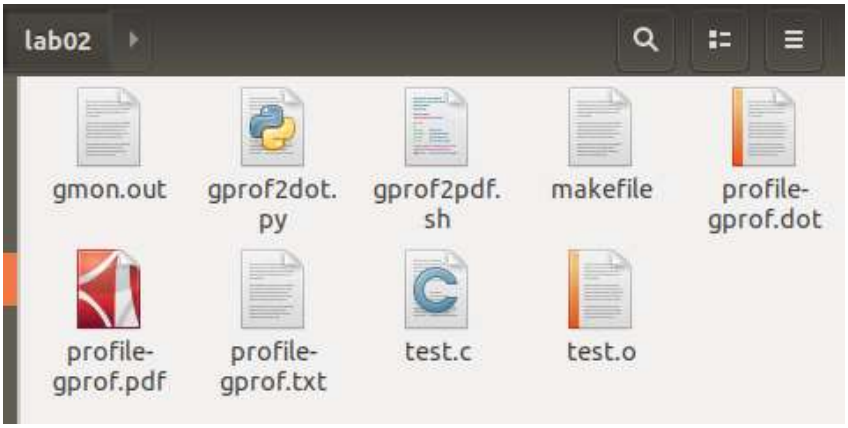
```
    for(;i<0xffffffff;i++); //some dummy delay

    for (i = 0; i < size; i++)
    {
        qout[i] = in1[i] / in2[i];
    }

}

int frequency_of_primes (int size) {
  int i,j;
  int n = size;
  int freq=n-1;
  for (i=2; i<=n; ++i) for (j=sqrt(i);j>1;--j) if (i%j==0) {--freq; break;}
  return freq;
}

int main(int argc,char **argv)
{
    int i, g;
    int size = 1e8;
    float *a = (float *)(malloc(size * sizeof(float )));
    float *b = (float *)(malloc(size * sizeof(float )));
    float *c = (float *)(malloc(size * sizeof(float )));
    float *d = (float *)(malloc(size * sizeof(float )));
    float *e = (float *)(malloc(size * sizeof(float )));
    float *f = (float *)(malloc(size * sizeof(float )));

    for (i = 0; i < size; i++)
    {
        e[i] = (i + 1.7);
        f[i] = (i + 1.7);
    }

    add(a,e,f,size);
    sub(b,e,f,size);
    muldiv(c,d,e,f,size);
        g = frequency_of_primes (99999);

    printf("sum = %f\n", e[0]*f[0]);
    printf ("The number of primes lower than 100,000 is: %d\n",g);
    free(a);
    free(b);
    free(c);
    free(d);
    free(e);
    free(f);

    return 0;
}
```

To execute the code All you have to do is write a command “make gprof”.

```
fasih@fasih-VirtualBox:~/lab02$ gedit test.c
fasih@fasih-VirtualBox:~/lab02$ make gprof
rm -f test test.o *~ perf.data gmon.out profile-gprof.* profile-perf.*
gcc -O1 -g -pg -fno-inline -fno-omit-frame-pointer -Wall -Wfatal-errors -I. -c
test.c -o test.o
gcc test.o -o test -L.  -lm -pg
./test
sum = 2.890000
The number of primes lower than 100,000 is: 9592
gprof -b test > profile-gprof.txt
bash ./gprof2pdf.sh profile-gprof.txt
fasih@fasih-VirtualBox:~/lab02$
```

It will show you the results but also created two files in directory (Flat profiling and call graph). One file contains readable data and others have graphical representation of the same data.

After refreshing your current directory, you will be able to see a pdf file named “profile-gprof.txt” and “profile-gprof.pdf”. Open both files.



Flat Profile:

```
Flat profile:

Each sample counts as 0.01 seconds.
  %   cumulative   self              self     total
 time   seconds   seconds    calls   s/call   s/call  name
 89.37     27.04    27.04        1    27.04    27.04  muldiv
  4.18     28.31     1.26                             main
  3.28     29.30     0.99        1     0.99     0.99  sub
  2.95     30.19     0.89        1     0.89     0.89  add
  0.53     30.35     0.16        1     0.16     0.16  frequency_of_primes
```

```
                     Call graph


granularity: each sample hit covers 2 byte(s) for 0.03% of 30.35 seconds

index % time    self  children    called     name
                                                 <spontaneous>
[1]    100.0    1.26   29.09                 main [1]
               27.04    0.00       1/1           muldiv [2]
                0.99    0.00       1/1           sub [3]
                0.89    0.00       1/1           add [4]
                0.16    0.00       1/1           frequency_of_primes [5]
-----------------------------------------------
               27.04    0.00       1/1           main [1]
[2]     89.1   27.04    0.00       1         muldiv [2]
-----------------------------------------------
                0.99    0.00       1/1           main [1]
[3]      3.3    0.99    0.00       1         sub [3]
-----------------------------------------------
                0.89    0.00       1/1           main [1]
[4]      2.9    0.89    0.00       1         add [4]
-----------------------------------------------
                0.16    0.00       1/1           main [1]
[5]      0.5    0.16    0.00       1         frequency_of_primes [5]
-----------------------------------------------

Index by function name

   [4] add                     [1] main                    [3] sub
   [5] frequency_of_primes     [2] muldiv
```

Here is what the fields in each line mean:

% time
: This is the percentage of the total execution time your program spent in this function. These should all add up to 100%.

cumulative seconds
: This is the cumulative total number of seconds the computer spent executing this functions, plus the time spent in all the functions above this one in this table.

self seconds
: This is the number of seconds accounted for by this function alone. The flat profile listing is sorted first by this number.

calls
: This is the total number of times the function was called. If the function was never called, or the number of times it was called cannot be determined (probably because the function was not compiled with profiling enabled), the *calls* field is blank.

self ms/call
: This represents the average number of milliseconds spent in this function per call, if this function is profiled. Otherwise, this field is blank for this function.

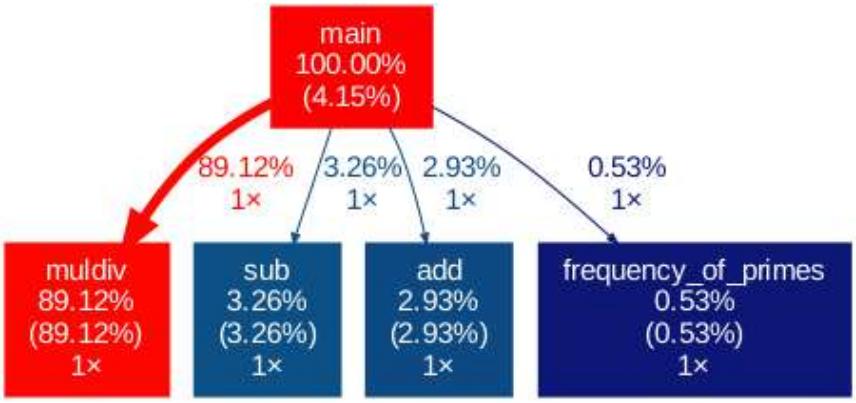
total ms/call
: This represents the average number of milliseconds spent in this function and its descendants per call, if this function is profiled. Otherwise, this field is blank for

this function. This is the only field in the flat profile that uses call graph analysis.

name

This is the name of the function. The flat profile is sorted by this field alphabetically after the *self seconds* and *calls* fields are sorted.

Call graph:



So, using Gprof method we can observe the execution time in the form of percentages of each function.

Example Code:

```c
#include<stdio.h>
int funcA()
{
int a=23;
int b=54;
for(int i=0x00;i<0xffffff;i++); // Delay Generation

return a+b;
}

int funcB(int x, int y)
{
for(int i = 0;i<0xfff;i++)
{
for(int i = 0;i<0xff;i++);
for(int i = 0;i<0xff;i++);
for(int i = 0;i<0xfff;i++);
}
return x*y;
}

int funcC()
{
int x=1153;
int y=2362;
for(int i=0;i<x;i++)
{
for(int j=0;j<y;j++);
}
return 0;
}

int main ()
{
int x=funcA();
printf("%d",x);
printf(" - - - ");
funcC();
x=funcC();
printf("%d",x);
x=funcB(11535,2362);
printf("%d",x);
return 0;
}
```

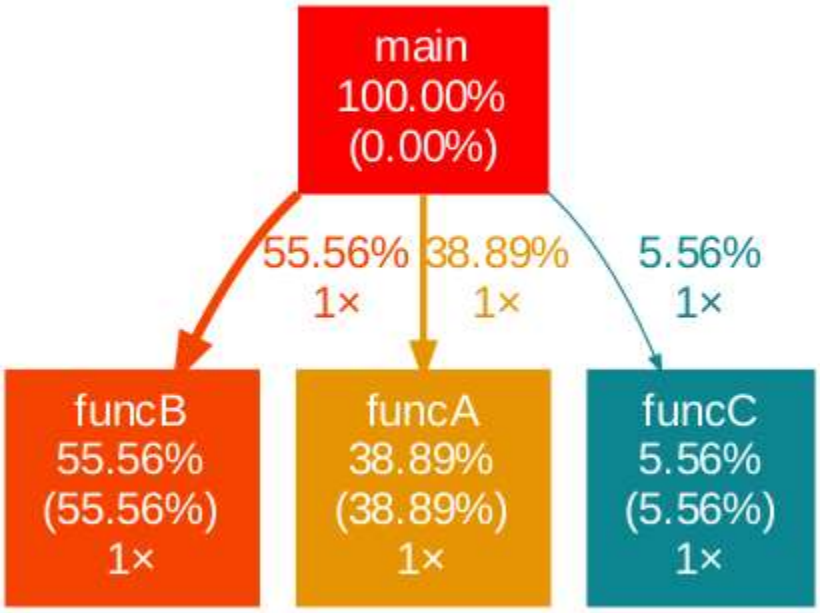
To execute the code All you have to do is write a command “make gprof”.

```
fasih@fasih-VirtualBox:~$ cd lab02
fasih@fasih-VirtualBox:~/lab02$ gedit test.c
fasih@fasih-VirtualBox:~/lab02$ make gprof
rm -f test test.o *~ perf.data gmon.out profile-gprof.* profile-perf.*
gcc -O1 -g -pg -fno-inline -fno-omit-frame-pointer -Wall -Wfatal-errors -I. -c
test.c -o test.o
gcc test.o -o test -L.  -lm -pg
./test
77 - - - 027245670gprof -b test > profile-gprof.txt
bash ./gprof2pdf.sh profile-gprof.txt
fasih@fasih-VirtualBox:~/lab02$ ls
gmon.out      gprof2pdf.sh  profile-gprof.dot  profile-gprof.txt  test.c
gprof2dot.py  makefile      profile-gprof.pdf  test               test.o
fasih@fasih-VirtualBox:~/lab02$ xdg-open profile-gprof.pdf
fasih@fasih-VirtualBox:~/lab02$
```

Flat Profile:

```
Flat profile:

Each sample counts as 0.01 seconds.
  %   cumulative   self              self     total
 time   seconds   seconds    calls  ms/call  ms/call  name
 55.83      0.10     0.10        1   100.50   100.50  funcB
 39.08      0.17     0.07        1    70.35    70.35  funcA
  5.58      0.18     0.01        1    10.05    10.05  funcC

                     Call graph

granularity: each sample hit covers 2 byte(s) for 5.53% of 0.18 seconds

index % time    self  children    called     name
                                                 <spontaneous>
[1]    100.0    0.00    0.18                 main [1]
                0.10    0.00       1/1           funcB [2]
                0.07    0.00       1/1           funcA [3]
                0.01    0.00       1/1           funcC [4]
-----------------------------------------------
                0.10    0.00       1/1           main [1]
[2]     55.6    0.10    0.00       1         funcB [2]
-----------------------------------------------
                0.07    0.00       1/1           main [1]
[3]     38.9    0.07    0.00       1         funcA [3]
-----------------------------------------------
                0.01    0.00       1/1           main [1]
[4]      5.6    0.01    0.00       1         funcC [4]

Index by function name

   [3] funcA                   [2] funcB                   [4] funcC
```

Call Graph:



Lab task:

1. Run the above example code according to the given instructions.

2. Write a factorial function and run through Gprof. And show the results of flat profile and call graph.

The End ☺